Automatikus skálázódás és terheléspróba bemutatása

A feladathoz a **Locust** terheléstesztelő keretrendszert használtam, aminek inicalizálásához a fényképalbum projekt mellé létrehoztam egy locustfile.py fájlt (forráskód a dokumentum végén, illetve a GitHub oldalon - https://github.com/Jona-G/felhoalapu-lab3 - itt a project mappában).

Az OpenShift felületen létrehoztam egy **Horizontal Pod Autoscaler**-t (HPA). A maximális podok számát 5-re állítottam, illetve ha egy pod eléri az 50%-os CPU kihasználtságot, akkor indít egy újat. Létrehoztam egy teszt felhasználót, amit a keretrendszer képes a bejelentkezésekhez felhasználni. Nagyobb súllyal állítottam be a főoldal megtekintését, ezzel szimulálva a nagyobb forgalmat.

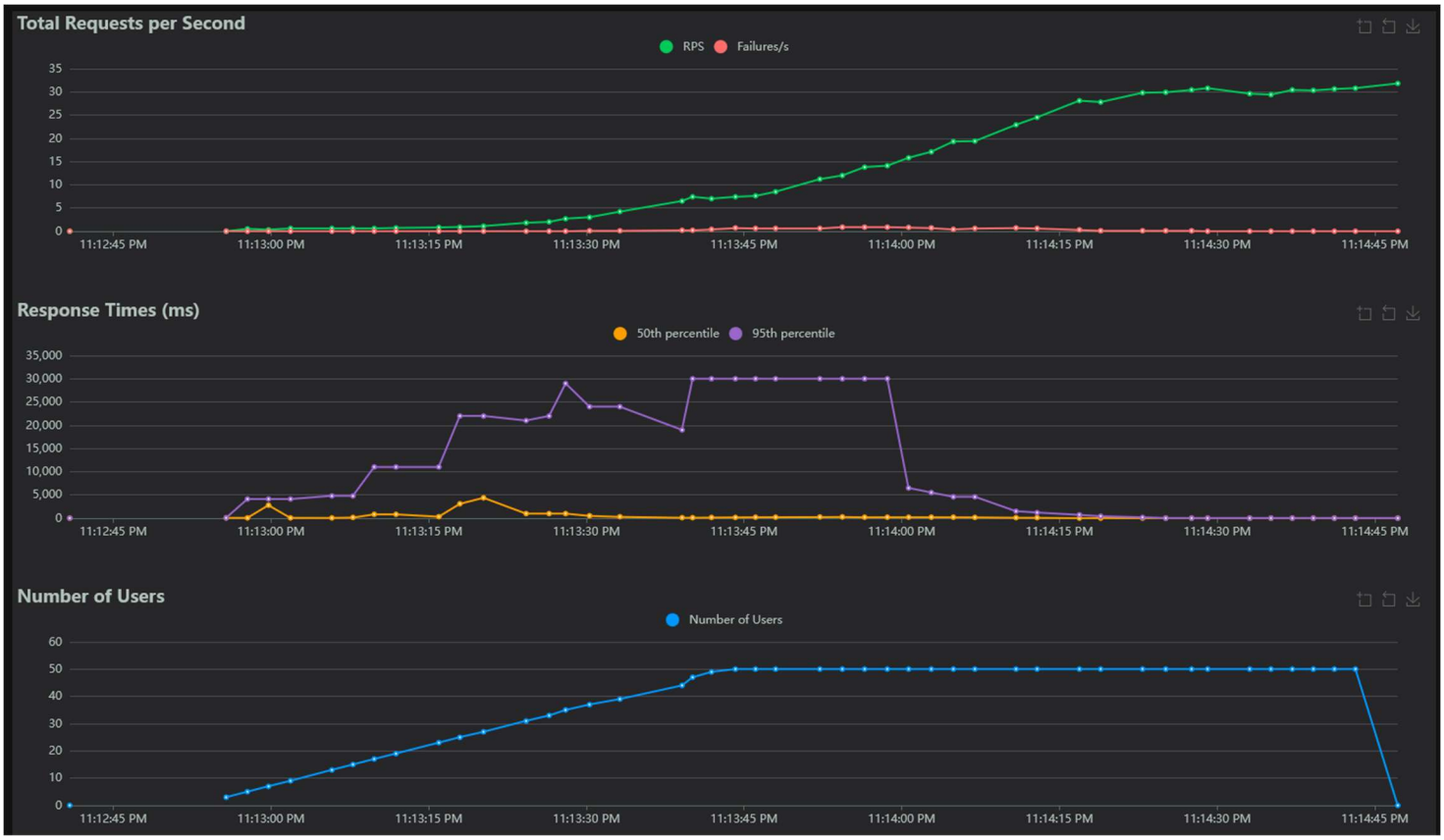
Létrehoztam egy ConfigMap erőforrást **locust-script** néven, ez lesz képes a locustfile.py tartalmát egy külön podban elindítani. A tesztelő frontend eléréséhez pedig egy Container Image-t hoztam létre **locust-tester** néven a **locustio/locust:latest** külső image felhasználásával. Végül, ezzel a pod-dal összekötöttem a locust-script-et, és a környezeti változók beállítása után már a felhasználói felület is elérhető lett.

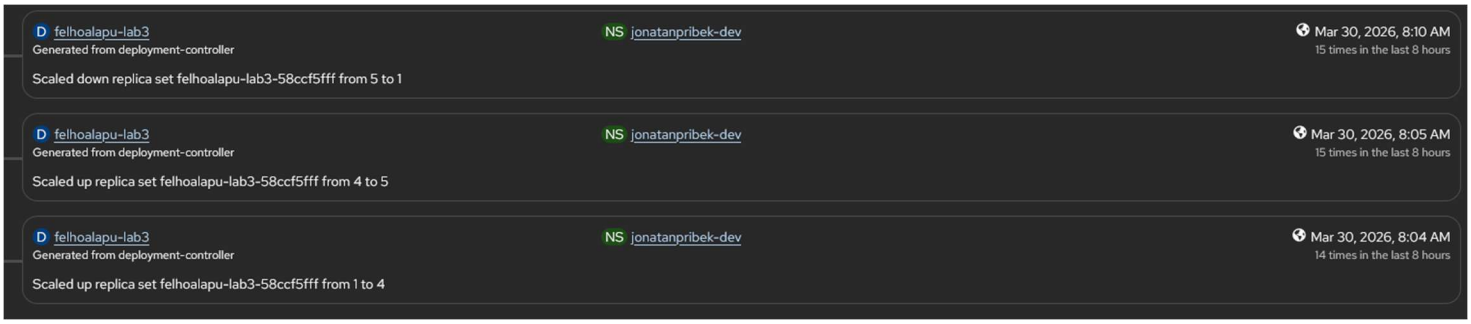
Itt egy 50 felhasználós szimulációt futtattam le, amit a rendszer másodpercenként egyesével vett be a tesztelésbe.

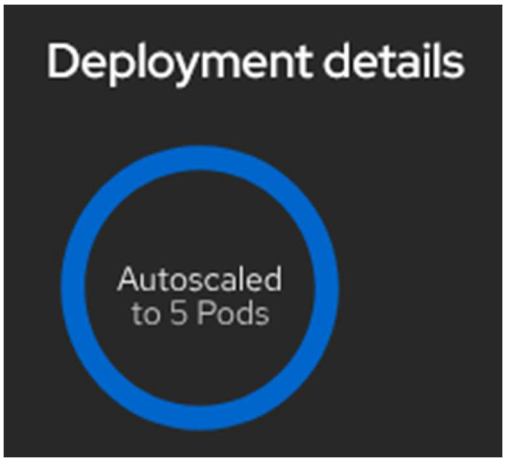
A tesztelés grafikonjai láthatóak a lejjebb látható első képen:

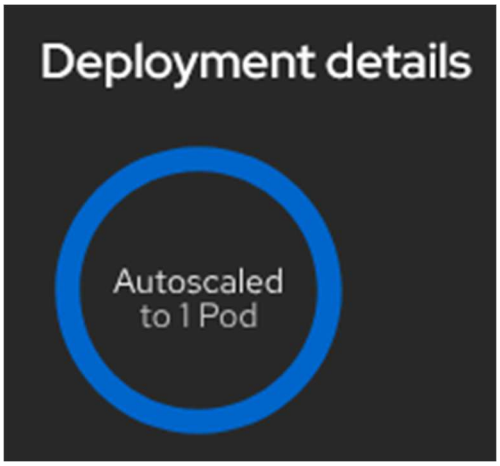
- Az első grafikon a másodpercenkénti kérések számának növekedését mutatja.
- A második grafikon a válaszidőt mutatja, ez az érték növekedett a felhasználók számával, ekkor lépett közbe a HPA, ami felvette a Pod-ok számát és ezt követően a válaszidő ismét lecsökkent.
- A harmadik grafikon a felhasználók számának egyenletes növekedését mutatja.

A tesztelés közben az OpenShift felületen is látható volt a Deployment részletei között a podok számának automatikus felskálázása, illetve a tesztelés után ezek visszavétele, ezek is láthatóak a lenti képeken.


Total Requests per Second
RPS
Failures/s
35
30
25
20
15
10
5
0
11:12:45 PM
11:13:00 PM
11:13:15 PM
11:13:30 PM
11:13:45 PM
11:14:00 PM
11:14:15 PM
11:14:30 PM
11:14:45 PM
Response Times (ms)
50th percentile
95th percentile
35,000
30,000
25,000
20,000
15,000
10,000
5,000
0
11:12:45 PM
11:13:00 PM
11:13:15 PM
11:13:30 PM
11:13:45 PM
11:14:00 PM
11:14:15 PM
11:14:30 PM
11:14:45 PM
Number of Users
Number of Users
60
50
40
30
20
10
0
11:12:45 PM
11:13:00 PM
11:13:15 PM
11:13:30 PM
11:13:45 PM
11:14:00 PM
11:14:15 PM
11:14:30 PM
11:14:45 PM


D felhoalapu-lab3
Generated from deployment-controller
NS jonatanpribek-dev
Mar 30, 2026, 8:10 AM
15 times in the last 8 hours
Scaled down replica set felhoalapu-lab3-58ccf5fff from 5 to 1
D felhoalapu-lab3
Generated from deployment-controller
NS jonatanpribek-dev
Mar 30, 2026, 8:05 AM
15 times in the last 8 hours
Scaled up replica set felhoalapu-lab3-58ccf5fff from 4 to 5
D felhoalapu-lab3
Generated from deployment-controller
NS jonatanpribek-dev
Mar 30, 2026, 8:04 AM
14 times in the last 8 hours
Scaled up replica set felhoalapu-lab3-58ccf5fff from 1 to 4


Deployment details
Autoscaled
to 5 Pods


Deployment details
Autoscaled
to 1 Pod

```python
from locust import HttpUser, task, between
import os

class GalleryUser(HttpUser):
    # rovid varakozas
    wait_time = between(1, 3)

    def on_start(self):
        # kapunk egy CSRF tokent
        response = self.client.get("/login/")
        self.csrftoken = response.cookies.get('csrftoken')

        # bejelentkezés
        self.client.post("/login/", {
            "username": "testuser",
            "password": "testpassword",
            "csrfmiddlewaretoken": self.csrftoken
        }, headers={"X-CSRFToken": self.csrftoken})

    @task(3)
    def view_gallery(self):
        self.client.get("/")

    @task(1)
    def upload_photo(self):
        # kep feltoltes szimulacio
        response = self.client.get("/upload/")
        csrftoken = response.cookies.get('csrftoken')

        dummy_image = ("test.jpg", b"dummy image data", "image/jpeg")

        self.client.post("/upload/",
            data={
                "name": f"loadtest_photo_{os.urandom(4).hex()}",
                "csrfmiddlewaretoken": csrftoken
            },
            files={"image": dummy_image},
            headers={"X-CSRFToken": csrftoken}
        )
```